

TECHNICAL ONBOARDING · INTERNAL USE

OryonX Technology Stack

A reference guide for new members of the technical team

PRODUCT	Ridley — multichannel AI assistant
MARKETS	Spain (Telnyx) and the UK (Vapi)
TEAM	René Medina · Diego Alejandro Julcamoro · Benjy Poli
DATE	July 2026

WELCOME

Welcome to the OryonX technical team

This document summarizes the complete OryonX technology stack so you can get oriented quickly: what each piece does, how they connect to each other, and where each responsibility lives. It doesn't replace the documentation in each repository, but it gives you the full map before you dive into the detail.

◆ What OryonX is

We build **Ridley**, a multichannel AI assistant (web chat, WhatsApp, voice) with an integrated CRM, sold as a managed service ("done-for-you") to service businesses — trades/construction, dental clinics, salons, real estate — in Spain and the UK. Our differentiation from self-serve tools (HubSpot, Zapier, GoHighLevel) is that **we build and run the house, we don't sell the hammer**.

The team

Person	Role	Main focus
René Medina	CEO & Founder	Sales, finance, investor relations
Diego Alejandro Julcamoro	CTO & Co-founder	Architecture, full-stack, product, security
Benjy Poli	Marketing & Social Media	Content, social, brand positioning

Architecture at a glance

The system is organized into four layers: **inbound channels** (voice, WhatsApp, web), **orchestration** (n8n connects everything), **business backend** (Laravel + PostgreSQL, multi-tenant), and **frontend** (React SSR). LLM inference is delegated to external providers — we don't train or host our own models.

Layer	Component	Role
Channels	Telnyx (ES voice) · Vapi (UK voice) · Twilio (UK WhatsApp/SMS) · Web chat	Point of contact with the end customer, 24/7
Voice AI	Vapi + Claude Haiku 4.5 + Cartesia TTS + Deepgram STT (UK) Telnyx AI Assistant Builder + Groq Llama 3.3-70b (ES)	Real-time conversational voice pipeline
Orchestration	n8n	Workflows: WhatsApp, lead qualification, notifications, email
Backend	Laravel 12 + PostgreSQL + Redis (Railway)	Multi-tenancy, CRM, business logic, rate limiting
Frontend	React + Vike SSR + Tailwind (Netlify)	Public site, service pages, landing pages

LAYER 1 · FRONTEND

React, Vike SSR and Tailwind

The frontend is the OryonX public website (oryonx.ai): the landing page, service pages, and the product mockups we show to prospective clients. It's deployed on Netlify.

Technology	Use at OryonX
React	Base UI component library
Vike (SSR)	Server-side rendering — improves SEO and initial load time
Tailwind CSS	Utility-first styling system; consistent design tokens
Inter (font)	Primary brand typeface
@unhead/react	Manages the HTML lang attribute (important for multi-language ES/UK SEO)
Netlify	Frontend hosting and continuous deployment

Design decisions already locked in

- Sustained dark navy background across the whole experience, with a single cyan accent.
- Amber is reserved exclusively for voice/urgency indicators — never mixed with the primary accent.
- Real product mockups instead of generic illustrations.
- One primary CTA per section, to avoid diluting conversion.

◆ On the horizon

Restructuring the "One system" grid (from 6 to 5 cards, aligned with the actual service pages) and building 5 /services/ pages with hero banners and entrance animations.

LAYER 2 · BACKEND

Laravel, PostgreSQL and multi-tenancy

The backend is the business brain: multi-tenancy, CRM, lead qualification, rate limiting, and all the logic that must not live duplicated in n8n. It runs on Railway.

Technology	Use at OryonX
Laravel 12	Main backend framework — orchestrates everything that isn't LLM inference
Eloquent ORM	Data access, relationships between models
PostgreSQL	Main database, multi-tenant
Redis	Caching and rate limiting (via Redis::incr() for atomic counting)
Railway	Hosting for Laravel, PostgreSQL and Redis

Multi-tenancy

Every OryonX client is an isolated tenant. This is implemented with **OrganizationScope**, an Eloquent global scope applied to the relevant tables, so every query is automatically filtered by organization without having to remember it in every controller.

Which modules and metrics are shown to each client is controlled by a dual-axis system: **plan** (Starter / Pro / Professional / Voice AI) and **business_model** (the client's vertical). This configuration lives centrally in config/plans.php and config/business_models.php — we avoid scattering permission logic across the codebase.

Important patterns to respect

- All business logic lives in Laravel, never duplicated in n8n.
- **LeadLookupController** is kept separate from the main **LeadController** for performance.
- **MetricsService** computes only the intersection of what the plan allows and what's relevant to the client's business_model.
- Phone numbers are normalized to E.164 format at every write path, via the **PhoneNormalizer** helper (a backfill command exists for legacy records).
- Rate limiting uses three escalating block tiers (10 min / 2 h / 24 h), persisted in the **RateLimitOffense** model.

◆ Why Laravel and not Django or another framework?

This architectural decision has already been validated: Laravel fits well with our **orchestrator** role (LLM inference is delegated to external services like OpenRouter, Groq and Vapi), has mature multi-tenancy support, and migrating to another framework would carry a high rewrite cost with no real benefit.

LAYER 3 · ORCHESTRATION

n8n — the connector for the whole system

n8n is the workflow automation layer: it connects inbound channels, the backend, the AI, and notification/email services. If something moves data between two systems, it probably goes through n8n.

WhatsApp pipeline (in production)

This is the most mature workflow in the stack. Its key pieces:

- **Delta-only** lead accumulation — only new changes are processed, not the full history.
- Parallel HTTP calls for rate-limit and lookup, to minimize latency.
- Duplicate detection via the **Merge_and_notify_duplicated** subworkflow.
- A "typing..." indicator so the conversation feels natural.
- A **/clear** command for development and testing.
- Returning-lead recognition, seeded from the existing CRM.

Lead qualification fields

name, company, challenge, client_volume, email, phone, language, suggested_plan, commercial_summary.

- The **suggested_plan** field must use lowercase values: starter, pro, professional, voice_ai.
- The **channel** field only accepts "phone" or "web_test" — never "voice".

Rules for editing n8n workflows

◆ Surgical edits, not rewrites

n8n JSON files are edited with Python/json scripts in a surgical way, validating connection integrity before delivery. Credentials, models, and unrelated nodes are **never** modified when applying a targeted fix.

LAYER 4 · VOICE AI

Ridley Voice — Spain and the United Kingdom

Ridley Voice has two distinct implementations depending on the market, for both technical reasons and differences in pipeline maturity.

Ridley Voice UK — Vapi pipeline

Piece	Provider
Voice orchestration	Vapi
Language model	Claude Haiku 4.5 (via OpenRouter)
Text-to-Speech	Cartesia Sonic
Speech-to-Text	Deepgram Nova-3
Telephony	Twilio (UK numbers)

Prompt engineering for this pipeline is at an advanced stage, with stress testing reaching ~8.5/10 quality. **Open issue:** audio degradation on phone calls — Telnyx records clean audio, but Vapi's recording comes out choppy. The SIP trunk configuration is suspected as the root cause; it hasn't been resolved yet. The proposed fix (not yet implemented) is manual IP-based SIP trunk configuration.

Ridley Voice ES — Telnyx pipeline

While the Vapi audio issue remains open, Spain uses Telnyx's **native AI Assistant Builder** as the interim production path, with **Groq Llama 3.3-70b-versatile** as the model. The prompt is compressed and hardened against several voice-specific issues:

- Phonetic mixing between languages.
- Correct spelling of acronyms.
- Anti-narration rules (the model should perform actions rather than describe what it's about to do).
- Correct sequencing of the call hangup.

◆ **Voice prompt engineering principles (apply to both pipelines)**

No markdown in voice prompts · numbered, sequential steps for the call hangup (prevents the model from skipping the farewell) · explicit anti-narration and anti-repetition rules with correct/incorrect examples · phonetic alphabet tables for Spanish TTS · SSML lang tags for proper names.

Voice AI cost base

Pipeline	Approximate cost
Vapi (Deepgram + model + Cartesia + platform + telephony)	\$0.09–0.12 / min all-in
Native Telnyx	\$0.07–0.08 / min

An 8–10% USD/GBP FX buffer is recommended when pricing in pounds.

AI, LLMS AND TELEPHONY

External providers we rely on

Language models

Model	Use	Via
Claude Haiku 4.5	Primary LLM for the system (text and UK voice)	OpenRouter
Groq Llama 3.3-70b-versatile	Production voice for the Spanish pipeline (Telnyx)	Groq

Telephony by market

Market	Provider	Use
Spain	Telnyx	Voice numbers, Ridley Voice ES
United Kingdom	Twilio	WhatsApp Business and SMS/voice
United Kingdom	Vapi	Voice AI orchestration

The UK Twilio number is connected to WhatsApp Business via Meta verification. Historical note: trial-account restrictions on Twilio were previously causing SMS/call issues — if you see odd behavior on a new number, check first whether the account is still on a trial plan.

Email and prospecting

Tool	Use
Amazon SES	Multi-client transactional email
Google Workspace	oryonx.ai domain, corporate email
Apollo.io	B2B prospecting (UK construction and dental)
Google Places API	Local micro-SMB prospecting by vertical (integrated with n8n + CRM)

DESIGN AND DEVELOPMENT TOOLS

How we work day to day

Design and asset generation

Tool	Use
Figma Make	UI prototyping and design
Nano Banana	AI image generation
OpenClaw / Claude Code	Primary development workflow for frontend changes

Typical workflow

- Diego works across the full stack: Laravel backend, React frontend, n8n configuration, voice prompt engineering, and product strategy.
- Complete, ready-to-apply artifacts (JSON files, PHP code, Claude Code prompts) are preferred over abstract explanations — the goal is for changes to be applied directly.
- QA runs in structured rounds, with turn-by-turn transcript analysis and numeric scoring.

◆ The current bottleneck

OryonX's main scalability constraint today is having a single technical person handling implementation, maintenance, support, and product development at the same time. Pre-built n8n templates and AI-accelerated builds partially mitigate this, but AI-generated systems that are similar but not identical across clients create future maintenance complexity. Hiring a second engineer is on the roadmap, currently blocked by capital.

QUICK REFERENCE

Full stack cheat sheet

Area	Technology
Frontend	React · Vike (SSR) · Tailwind CSS · Inter · Netlify
Backend	Laravel 12 · Eloquent · PostgreSQL · Redis · Railway
Orchestration	n8n
Primary LLM	Claude Haiku 4.5 (via OpenRouter)
Voice — UK	Vapi · Cartesia Sonic (TTS) · Deepgram Nova-3 (STT) · Twilio
Voice — Spain	Telnyx AI Assistant Builder · Groq Llama 3.3-70b-versatile
Telephony	Telnyx (ES) · Twilio (UK WhatsApp/SMS) · Vapi (UK voice)
Email	Amazon SES · Google Workspace
Prospecting	Apollo.io · Google Places API
Design/Dev	Figma Make · Nano Banana · OpenClaw / Claude Code

Suggested next steps for you

- Request access to Railway, Netlify, n8n, Telnyx, Vapi, and the main repository.
- Review the WhatsApp workflow in n8n — it's the most mature and a good entry point for understanding the pattern used across the whole system.
- Read config/plans.php and config/business_models.php to understand the per-tenant access model.
- If you'll be touching voice, get familiar with the prompt engineering rules before editing any production prompt.

This document is a snapshot of the stack as of July 2026. The system evolves fast — if anything here doesn't match what you see in the code, the code wins. Update this document when you find discrepancies.